

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

1. Título

Informe técnico del Desarrollo de Backend Proformax

2. Introducción

El presente informe documenta el proceso de desarrollo del backend del sistema Proformax, correspondiente a la fase de implementación de los servicios, reglas de negocio y mecanismos de acceso a datos que soportan las funcionalidades principales de la plataforma. El trabajo fue estructurado siguiendo una arquitectura en capas, con el objetivo de mantener una separación clara entre rutas, controladores, servicios y repositorios, favoreciendo la mantenibilidad, escalabilidad y trazabilidad del sistema.

La aplicación backend fue desarrollada sobre Node.js utilizando Express.js como framework principal, Prisma como ORM para la persistencia de datos y PostgreSQL como sistema gestor de base de datos. Asimismo, se incorporaron mecanismos de seguridad basados en autenticación JWT, validación de entradas, control de acceso por roles, manejo centralizado de errores, registro de eventos y documentación interactiva mediante Swagger. Este módulo constituye el núcleo funcional de Proformax, administrando entidades como usuarios, clientes, productos, proveedores, proformas, facturas y configuraciones del sistema.

El propósito de este informe es evidenciar el cumplimiento de las actividades técnicas necesarias para consolidar un backend funcional, seguro y organizado, alineado con buenas prácticas de desarrollo de software y orientado a facilitar la operación del sistema en un entorno real.

3. Objetivos

3.1 Objetivo General

Desarrollar e implementar el backend del sistema Proformax aplicando una arquitectura en capas, mecanismos de seguridad basados en autenticación JWT, validación de datos y buenas prácticas de desarrollo, con el fin de garantizar la integridad, trazabilidad, escalabilidad y disponibilidad de la información gestionada por la plataforma.

3.2 Objetivos Específicos

- **Objetivo 1:** Diseñar y estructurar la lógica del backend separando responsabilidades entre controladores, servicios, repositorios y rutas, para facilitar el mantenimiento y la evolución del sistema.
- **Objetivo 2:** Implementar servicios REST para la gestión de usuarios, clientes, productos, proveedores, proformas y facturas, asegurando un acceso eficiente y ordenado a los datos mediante PostgreSQL y Prisma.
- **Objetivo 3:** Incorporar mecanismos de seguridad, documentación y control de acceso mediante JWT, validaciones de entrada, manejo centralizado de errores y

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

Swagger, con el propósito de mejorar la confiabilidad y facilitar las pruebas del sistema.

4. Alcance

El presente documento abarca las actividades realizadas durante el desarrollo del backend del sistema Proformax, incluyendo:

- Diseño e implementación de la arquitectura backend en capas, organizada en rutas, controladores, servicios y repositorios.
- Desarrollo de servicios REST para la gestión de usuarios, clientes, productos, proveedores, proformas, facturas y configuraciones del sistema.
- Implementación de la capa de persistencia de datos mediante Prisma y PostgreSQL.
- Configuración de mecanismos de seguridad basados en autenticación JWT y control de acceso por roles.
- Incorporación de validaciones de entrada, manejo centralizado de errores y respuesta estructurada de la API.
- Generación de documentación interactiva de la API mediante Swagger/OpenAPI.
- Implementación de lógica de negocio orientada a garantizar integridad, trazabilidad y consistencia en el tratamiento de los datos.

Quedan fuera del alcance del presente trabajo:

- Las pruebas unitarias, de integración y validación funcional del sistema.
- La integración con el frontend o capa de presentación.
- El despliegue en ambiente de producción, incluyendo infraestructura, configuración de servidores y publicación del sistema.
- La automatización de procesos de CI/CD y administración de entornos de despliegue.
- El desarrollo de funcionalidades adicionales no contempladas en el backend actual.

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

5. Definiciones

Término	Definición
Node.js	Entorno de ejecución de JavaScript del lado del servidor, basado en el motor V8 de Google Chrome, utilizado para construir el backend del sistema Proformax.
Backend	Conjunto de componentes del sistema responsables de la lógica de negocio, la persistencia de datos y la exposición de servicios (endpoints) que consume el frontend.
API RESTful	Interfaz que permite la comunicación entre cliente y servidor mediante el protocolo HTTP, siguiendo los principios REST (recursos, verbos HTTP, representaciones y estados).
JWT (JSON Web Token)	Formato de token firmado digitalmente que transporta información de identidad y se utiliza para autenticación sin estado; se envía normalmente en el encabezado.
Prisma ORM	Herramienta de mapeo objeto-relacional para Node.js que facilita la definición del esquema de base de datos, migraciones y acceso a datos de forma tipada.
Express.js	Framework minimalista para Node.js que facilita la creación de rutas, middlewares y controladores HTTP.
Variable de entorno	Par clave-valor definido fuera del código fuente que permite configurar el comportamiento de la aplicación según el entorno de ejecución (desarrollo, pruebas, producción).
Middleware	Función intermedia en la cadena de manejo de solicitudes que puede validar, transformar, autenticar o autorizar peticiones antes de que lleguen al controlador final.
Migración de BD	Proceso controlado de versionar y aplicar cambios en el esquema de la base de datos utilizando Prisma Migrate.
Swagger / OpenAPI	Especificación y conjunto de herramientas para definir, documentar y probar de forma interactiva los endpoints REST de la API.

Cuadro 1: Glosario de términos técnicos

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

6. Normativa Legal y Técnica Aplicable

El desarrollo, despliegue y operación del backend del proyecto se realiza atendiendo a normas, buenas prácticas y marcos de control que garantizan la calidad del software, la seguridad de la información, la protección de datos personales y la continuidad del servicio. A continuación se describen los estándares y referencias principales que son aplicables o recomendables para este proyecto:

- **Gestión de la calidad del software (ISO 9001:2015):** Lineamientos para la gestión de calidad del proceso de desarrollo, control documental, verificación de requisitos funcionales y no funcionales y mejora continua del ciclo de vida del software.
- **Gestión de la seguridad de la información (ISO/IEC 27001 / ISO/IEC 27002):** Marco para establecer, operar y mejorar un Sistema de Gestión de Seguridad de la Información (SGSI). Incluye controles para clasificación de activos, control de acceso, gestión de incidentes, cifrado, y políticas de respaldo y continuidad.
- **Privacidad y protección de datos personales (GDPR / normativa local aplicable):** Principios de tratamiento de datos (licitud, finalidad, minimización, exactitud, limitación de almacenamiento, integridad y confidencialidad). Aplicable tanto el Reglamento General de Protección de Datos (GDPR) como la legislación nacional o sectorial pertinente; se recomienda documentar bases legales, términos de consentimiento y acuerdos de tratamiento con terceros.
- **Seguridad de aplicaciones web (OWASP ASVS OWASP Top Ten):** Requisitos y controles para el desarrollo seguro (validación de entradas, control de autenticación y sesiones, gestión de errores, políticas de seguridad de cabeceras, protección frente a inyección, XSS, CSRF, etc.). OWASP ASVS sirve como referencia para verificar el nivel de seguridad de la aplicación.
- **Autenticación y gestión de identidades (NIST SP 800-63 / buenas prácticas):** Recomendaciones para gestión de autenticación digital, requisitos de verificación de identidad, políticas de contraseñas, multifactor (MFA) y emisión/gestión de tokens (p. ej. JWT) de forma segura.
- **Criptografía y transporte seguro (TLS / NIST / RFCs relevantes):** Empleo de algoritmos y parámetros criptográficos recomendados para cifrado en tránsito (TLS 1.2+ / 1.3), almacenamiento de secretos y gestión de claves. Uso de bibliotecas y procedimientos que cumplan con las guías de la organización para proteger datos sensibles.
- **Control y gestión de vulnerabilidades / ciclo de vida seguro (SDL):** Integración de análisis estático y dinámico, escaneo de dependencias, gestión de parches y políticas de respuesta ante vulnerabilidades (CVE tracking). Automatizar pruebas de seguridad en CI/CD cuando sea posible.
- **Continuidad del negocio y respuesta ante incidentes (ISO 22301 / NIST SP 800-34 / NIST SP 800-61):** Planes de recuperación ante desastres, continuidad operacional, copias de seguridad, RTO/RPO definidos y procedimientos

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

de gestión de incidentes.

- **Gestión de riesgos (ISO 31000 / COSO ERM):** Evaluación, tratamiento y seguimiento de riesgos tecnológicos y de negocio; priorización de controles y mitigaciones basadas en impacto y probabilidad.
- **Requisitos contractuales y normativos sectoriales:** Cumplimiento de obligaciones derivadas de contratos con clientes, SLAs y normativas sectoriales específicas que puedan imponer requisitos adicionales sobre auditoría, retención de registros o acceso a la información.
- **Cumplimiento y auditoría (SOC 2 / ISO 27001 audits):** Implantación de evidencias, registros de control y procedimientos que permitan auditorías internas y externas sobre la eficacia de los controles aplicados.
- **Formación y competencias (ISO 10015 / buenas prácticas de capacitación):** Capacitación continua del equipo en seguridad, desarrollo seguro y manejo de datos personales para mantener competencias alineadas con las obligaciones normativas y las mejores prácticas.

7. Método / Políticas / Contenido

7.1 Método

El desarrollo del backend se ejecutó siguiendo el enfoque ágil definido por la Guía Scrum y los principios de gestión de calidad de la ISO 9001:2015 [1], aplicando iteraciones planificadas a través de sprints que permitieron la entrega incremental de funcionalidades. Los métodos utilizados para garantizar la calidad y cumplimiento técnico incluyeron:

- **Arquitectura en capas:** Aplicación de la estructura propuesta en la documentación técnica (controladores, servicios, repositorios y entidades), asegurando separación de responsabilidades y mantenibilidad del código.
- **Control de versiones y trazabilidad:** Uso de Gitflow para la gestión de ramas y control de cambios, manteniendo la trazabilidad de los commits y las historias de usuario.
- **Validación continua:** Pruebas de endpoints y verificación de integridad de la base de datos mediante Postman y Swagger/OpenAPI.
- **Gestión de riesgos técnicos:** Aplicación del marco de referencia de la ISO 31000:2018 [14] para identificar, evaluar y mitigar riesgos en la seguridad del sistema y dependencias externas.
- **Seguridad por diseño:** Implementación de mecanismos de cifrado, control de acceso, roles y auditoría de logs conforme a los lineamientos de la ISO/IEC 27001:2022 [2] y las recomendaciones de la OWASP ASVS 4.0 [7].

7.2 Políticas

Las políticas establecidas durante el desarrollo del backend de Proformax se definieron para asegurar la conformidad normativa, la protección de la información comercial

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

y la trazabilidad del producto de software:

1. **Política de seguridad de la información:** Todo componente backend debe proteger los datos comerciales, de inventario y de facturación almacenados y transmitidos, aplicando la gestión segura de variables de entorno, control de accesos basado en roles (RBAC) y validación de tokens JWT, conforme a las normas ISO/IEC 27001:2022 [4] e ISO/IEC 27002:2022 [5].
2. **Política de privacidad y confidencialidad de datos:** El tratamiento de la información personal de clientes y proveedores se realiza en estricto cumplimiento de la Ley Orgánica de Protección de Datos Personales (LOPD) del Ecuador [1] y los principios del Convenio 108+, asegurando la licitud, proporcionalidad, minimización de datos y protección contra accesos no autorizados.
3. **Política de control de calidad técnica:** La integración de nuevas funcionalidades, las modificaciones en el esquema de la base de datos (a través de migraciones controladas en Prisma ORM) y la actualización de la documentación de la API requieren revisiones de código y pruebas de endpoints, alineándose a los principios de mejora continua de la ISO 9001:2015 [1].
4. **Política de gestión de la configuración segura:** Para mitigar riesgos de exposición (Security Misconfiguration), queda estrictamente prohibido el versionado de credenciales reales en el código fuente. La infraestructura y los secretos se gestionarán cumpliendo con las directrices de OWASP Top 10:2021 y NIST SP 800-204C [8].
5. **Política de gestión de riesgos e integraciones:** Cualquier cambio arquitectónico o conexión con servicios de terceros (como el procesamiento externo a través de la API de Azure) debe ser evaluado previamente para identificar y mitigar vulnerabilidades de seguridad y disponibilidad, basándose en el marco de la ISO 31000:2018 [14].
6. **Política de continuidad de servicios:** El diseño de la arquitectura y la conexión con la base de datos PostgreSQL deben garantizar el acceso concurrente estable a los módulos críticos (cotizaciones y facturación), previendo mecanismos de recuperación ante fallos de acuerdo con los lineamientos de continuidad de negocio de la ISO 22301:2019 [15].

7.3 Contenido

7.3.1 Estructura del Proyecto (Backend)

El backend se organiza por responsabilidades. En la raíz se separan los principales componentes del sistema para facilitar el mantenimiento y la navegación del código:

- **src/:** código fuente de la aplicación (rutas, controladores, servicios y repositorios).
- **prisma/:** definición del schema, migraciones y seed de la base de datos.
- **logs/:** archivos de registros generados por el sistema.

La distribución interna de **src/** se explica en la sección de arquitectura en capas

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

y la estructura base completa se muestra más adelante.

7.3.2 Visión General

- **Base URL:** /api/v1
- **Documentación Swagger:** /api/docs y spec JSON en /api/docs.json
- **Health check:** /api/v1/health

7.3.3 Arquitectura en Capas

El backend sigue una arquitectura en capas para separar responsabilidades y facilitar el mantenimiento:

- **Entrada HTTP / Rutas (src/routes):** define endpoints, aplica validaciones y middlewares por ruta.
- **Controladores (src/controllers):** adaptan la solicitud, coordinan el flujo y construyen la respuesta.
- **Servicios (src/services):** contienen la lógica de negocio y reglas del dominio.
- **Repositorios (src/repositories):** encapsulan el acceso a datos usando Prisma.
- **Persistencia (prisma/):** schema, migraciones y seed de base de datos.
- **Middlewares (src/middlewares):** autenticación, autorización, validaciones y manejo de errores.
- **Configuración (src/config):** conexiones, logger, Swagger, integraciones externas.
- **App y arranque (src/app.js, src/index.js):** config de Express, middlewares globales y puesta en marcha.

Estructura base de carpetas:

```
prisma/
  schema.prisma
  migrations/
  seed.js
src/
  app.js
  index.js
  config/
  controllers/
  middlewares/
  repositories/
  routes/
  services/
logs/
```

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

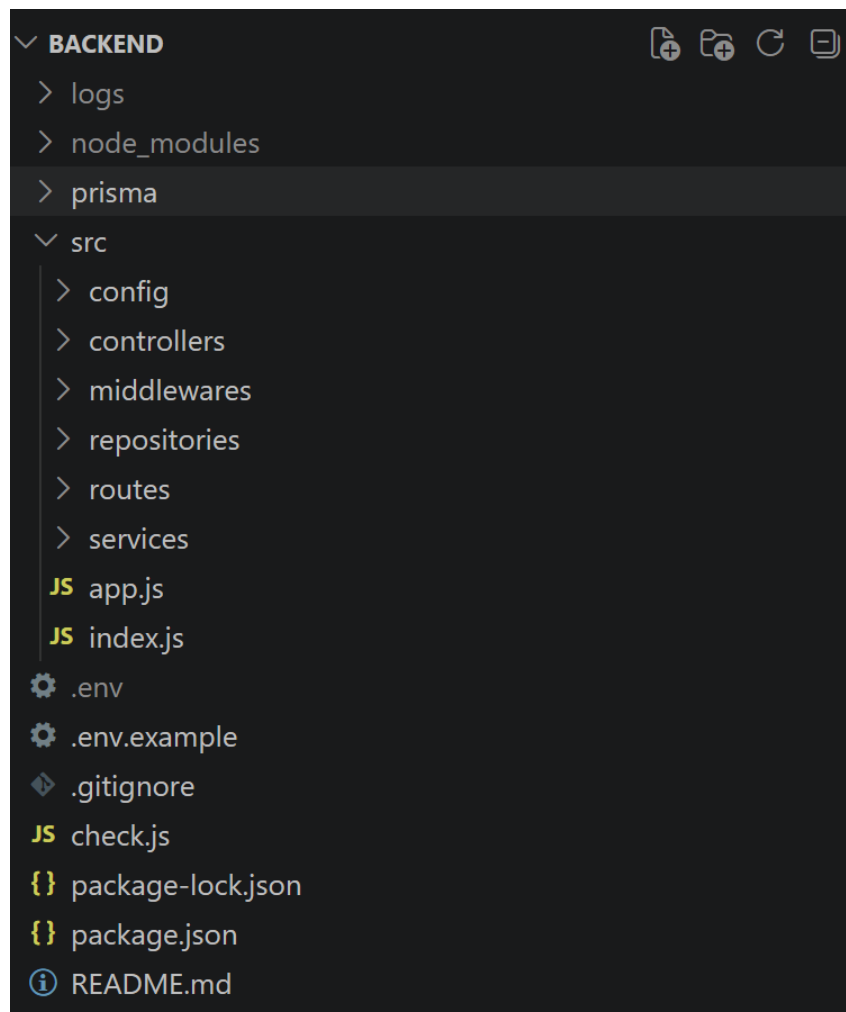


Figura 1: Estructura de carpetas del backend de Proformax

7.3.4 Módulos de la API

Auth

Qué hace: autentica usuarios y entrega JWT.

Ejemplo de endpoint: POST /api/v1/auth/login

Ejemplo de entrada:

```
{
  "username": "admin",
  "password": "Admin@1234"
}
```

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

Auth Autenticación y gestión de sesión ^		
POST	/auth/login	Iniciar sesión
POST	/auth/logout	Cerrar sesión
GET	/auth/me	Perfil del usuario autenticado

Figura 2: Documentación en Swagger - Módulo Auth

Usuarios

Qué hace: administra usuarios del sistema (solo rol ADMIN).

Ejemplo de endpoint: POST /api/v1/usuarios

Ejemplo de entrada:

```
{
  "username": "vendedor01",
  "password": "Passw0rd!",
  "rol": "vendedor"
}
```

Usuarios Administración de usuarios del sistema (requiere rol ADMIN) ^		
GET	/usuarios	Listar usuarios
POST	/usuarios	Crear usuario
GET	/usuarios/{id}	Obtener usuario por ID
PUT	/usuarios/{id}	Actualizar datos de usuario
PATCH	/usuarios/{id}/estado	Activar o desactivar usuario

Figura 3: Documentación en Swagger - Módulo Usuarios

Productos

Qué hace: gestiona productos y servicios del inventario.

Ejemplo de endpoint: POST /api/v1/productos

Ejemplo de entrada:

```
{
  "codigo": "PRD-001",
  "nombre": "Piso de madera",
  "descripcion": "Parquet de roble",
  "tipo": "producto",
  "precioBase": 25.50,
  "stockActual": 100,
  "aplicaIva": true
}
```

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

Productos Gestión de productos y servicios (inventario)		
GET	/productos Listar productos	🔒 ▼
POST	/productos Crear producto	🔒 ▼
GET	/productos/{id} Obtener producto por ID	🔒 ▼
PUT	/productos/{id} Actualizar producto completo	🔒 ▼
PATCH	/productos/{id} Actualizar producto parcialmente	🔒 ▼
DELETE	/productos/{id} Desactivar producto (baja lógica)	🔒 ▼

Figura 4: Documentación en Swagger - Módulo Productos

Proformas

Qué hace: crea y gestiona proformas comerciales con detalle de ítems.

Ejemplo de endpoint: POST /api/v1/proformas

Ejemplo de entrada:

```
{
  "clienteId": 1,
  "fechaValidez": "2026-05-30",
  "porcentajeDescuento": 5,
  "observaciones": "Entrega en 10 días",
  "detalles": [
    {
      "productoServicioId": 3,
      "cantidad": 10,
      "precioUnitario": 45.25
    }
  ]
}
```

Proformas Creación y gestión del ciclo de vida de proformas comerciales		
GET	/proformas Listar proformas	🔒 ▼
POST	/proformas Crear proforma	🔒 ▼
GET	/proformas/{id} Obtener proforma por ID	🔒 ▼
PUT	/proformas/{id} Actualizar proforma	🔒 ▼
GET	/proformas/{id}/pdf Exportar proforma como PDF	🔒 ▼
PATCH	/proformas/{id}/estado Cambiar estado de proforma	🔒 ▼

Figura 5: Documentación en Swagger - Módulo Proformas

Proveedores

Qué hace: administra proveedores y datos de facturación.

Ejemplo de endpoint: POST /api/v1/proveedores

Ejemplo de entrada:

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

```
{
  "identificacion": "1799999999001",
  "razonSocial": "Maderas Andinas S.A.",
  "nombreComercial": "Maderas Andinas",
  "direccion": "Av. Principal 123",
  "telefono": "+593 99 999 9999",
  "email": "ventas@maderas.com"
}
```

Proveedores Gestión de proveedores			^
GET	/proveedores	Listar proveedores	🔒 ▼
POST	/proveedores	Crear proveedor	🔒 ▼
GET	/proveedores/{id}	Obtener proveedor por ID	🔒 ▼
PUT	/proveedores/{id}	Actualizar proveedor completo	🔒 ▼
PATCH	/proveedores/{id}	Actualizar proveedor parcialmente	🔒 ▼
DELETE	/proveedores/{id}	Desactivar proveedor (baja lógica)	🔒 ▼

Figura 6: Documentación en Swagger - Módulo Proveedores

Cientes

Qué hace: gestiona clientes y su información de contacto.

Ejemplo de endpoint: POST /api/v1/clientes

Ejemplo de entrada:

```
{
  "identificacion": "1723456789",
  "nombres": "Juan",
  "apellidosRazonSocial": "Perez",
  "email": "juan.perez@example.com",
  "telefono": "0999999999",
  "direccion": "Calle 10 y Av 3"
}
```

Clientes Gestión de clientes			^
GET	/clientes	Listar clientes	🔒 ▼
POST	/clientes	Crear cliente	🔒 ▼
GET	/clientes/{id}	Obtener cliente por ID	🔒 ▼
PUT	/clientes/{id}	Actualizar cliente (reemplazo completo)	🔒 ▼
PATCH	/clientes/{id}	Actualizar cliente parcialmente	🔒 ▼
DELETE	/clientes/{id}	Eliminar cliente	🔒 ▼

Figura 7: Documentación en Swagger - Módulo Clientes

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

Configuración

Qué hace: administra la configuración de la empresa (RUC, IVA, datos generales).

Ejemplo de endpoint: PATCH /api/v1/config

Ejemplo de entrada:

```
{
  "porcentajeIvaVigente": 15.0
}
```

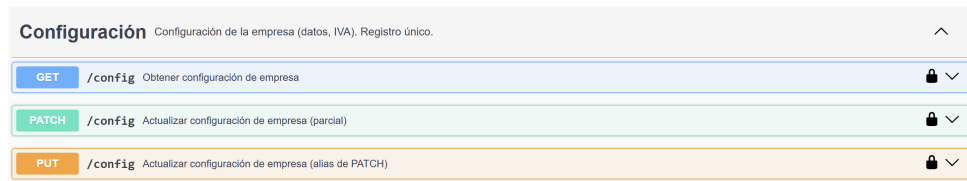


Figura 8: Documentación en Swagger - Módulo Configuración

Facturas

Qué hace: analiza y registra facturas de compra, integrando Azure AI Document Intelligence.

Ejemplo de endpoint: POST /api/v1/facturas/confirmar

Ejemplo de entrada:

```
{
  "proveedorId": 1,
  "numeroFactura": "001-001-000099930",
  "fechaEmision": "2026-04-21",
  "total": 400.00,
  "items": [
    {
      "descripcion": "Tablones roble",
      "cantidad": 20,
      "precioUnitario": 15.00,
      "totalItem": 300.00,
      "productoId": 5
    },
    {
      "descripcion": "Barniz",
      "cantidad": 10,
      "precioUnitario": 10.00,
      "totalItem": 100.00
    }
  ]
}
```

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

Facturas Gestión de facturas de compra con Azure AI Document Intelligence. Flujo de dos pasos para garantizar asociación correcta de proveedor.		
GET	/facturas Listar facturas de compra	🔒 ▼
GET	/facturas/{id} Obtener factura por ID con detalles	🔒 ▼
POST	/facturas/analizar [Paso 1] Analizar factura con Azure AI (sin guardar)	🔒 ▼
POST	/facturas/confirmar [Paso 2] Confirmar y guardar factura	🔒 ▼

Figura 9: Documentación en Swagger - Módulo Facturas

7.3.5 Seguridad y Protección de Rutas

- **Autenticación JWT:** el login entrega un token que se envía en `Authorization: Bearer <token>`.
- **Protección por rol:** se usan middlewares `authenticate` y `authorize` para restringir rutas. Ejemplos:
 - Usuarios, Productos y cambios de Configuración requieren rol ADMIN.
 - Proformas permite ADMIN y vendedor para crear/actualizar.
 - Facturas restringe análisis/confirmación a ADMIN.
- **Rate limiting:** límite general en `/api/` y límite estricto en `/api/v1/auth` para mitigar fuerza bruta.
- **Seguridad de cabeceras:** Helmet configura CSP y otras cabeceras de seguridad.
- **CORS controlado:** orígenes permitidos por variable de entorno.
- **Validación de entradas:** reglas con `express-validator` antes de llegar a los controladores.
- **Uploads seguros:** facturas aceptan PDF/JPEG/PNG/TIFF con límite de 10 MB.

7.3.6 Integración con Azure AI para Lectura de Facturas y Actualización Automática de Stock

Objetivo

Este módulo gestiona el flujo de lectura automatizada de facturas de compra físicas o digitales utilizando Azure Document Intelligence, culminando con la actualización automática del stock en inventario una vez que la factura es confirmada por el usuario.

Componentes Involucrados

- **Configuración Azure:** inicializa el cliente del SDK y valida credenciales en `src/config/azureDocumentAI.js`.
- **Rutas HTTP:** expone los endpoints de análisis y confirmación en `src/routes/factura.routes`.
- **Controlador:** recibe la petición y delega la ejecución al servicio en `src/controllers/facturaC`.
- **Servicio:** orquesta el flujo, aplica reglas de negocio y validaciones en `src/services/facturaCom`.
- **Repositorio:** ejecuta el análisis contra la API de Azure y realiza las operaciones transaccionales en la base de datos en `src/repositories/facturaCompra.repository.js`.

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

Variables de Entorno Requeridas

Para el correcto funcionamiento del servicio, es indispensable definir las siguientes variables. Si estas no existen, el backend retornará un error al intentar procesar un documento:

- AZURE_FORM_RECOGNIZER_ENDPOINT
- AZURE_FORM_RECOGNIZER_API_KEY

Flujo General de Procesamiento

1. **Analizar:** se sube el documento de la factura (PDF o imagen) y se extraen los datos crudos mediante Azure. En esta fase no se guarda información en la base de datos.
2. **Confirmar:** el usuario revisa y confirma el proveedor y los datos extraídos. El sistema procede a guardar el registro y aplicar la actualización de inventario.

Paso 1: Analizar Factura (Azure AI)

Endpoint: POST /api/v1/facturas/analizar

Entrada (multipart/form-data): Campo factura con el archivo PDF/JPEG/PNG/TIFF (Tamaño máximo: 10 MB).

Proceso Interno:

- Multer carga el archivo en memoria y valida su tipo y tamaño.
- El repositorio invoca el modelo **prebuilt-invoice** de Azure Document Intelligence.
- Se normalizan los campos críticos (proveedor, número de comprobante, fechas, totales e ítems).
- El servicio genera candidatos de proveedor (haciendo *matching* con la base de datos) y levanta avisos de calidad si hay inconsistencias.

Salida Principal:

- **datosExtraidos:** información estructurada detectada.
- **candidatosProveedor:** lista ordenada por nivel de coincidencia (*score* 100 para RUC exacto, 70 para nombre parcial, 40 para palabras clave).
- **avisos:** advertencias generadas cuando faltan datos obligatorios o la confianza de la IA es baja.

Paso 2: Confirmar Factura (Guardar y Actualizar Stock)

Endpoint: POST /api/v1/facturas/confirmar

Ejemplo de Entrada:

```
{
  "proveedorId": 1,
  "numeroFactura": "001-001-000099930",
  "fechaEmision": "2026-04-21",
  "total": 400.00,
  "items": [
```

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

```
{
  "descripcion": "Tablones roble",
  "cantidad": 20,
  "codigoProducto": "PRD-001",
  "precioUnitario": 15.00,
  "totalItem": 300.00,
  "productoId": 5
}
```

Reglas y Matching:

- El `proveedorId` debe existir previamente en la base de datos.
- El `numeroFactura` debe ser único para evitar duplicidades.
- Las cantidades de los ítems se redondean a enteros para la gestión de stock.
- Si se envía el `productoId`, se asocia directamente. En caso contrario, el sistema intenta un *match* por `codigoProducto` o `descripcion`. Los ítems sin correspondencia se reportan como texto libre y no afectan el inventario.

Actualización Automática de Stock

La confirmación ejecuta una **transacción atómica** en Prisma que garantiza la consistencia de los datos:

1. Creación de la cabecera en la tabla `facturas_compra`.
2. Inserción de los registros en `detalles_compra`.
3. Incremento del campo `stockActual` en la tabla `productos_servicios` (`stockActual += cantidad`) por cada ítem reconocido. Si no se reconocen productos válidos, la factura se registra sin alterar el inventario.

Control de Errores y Consideraciones Operativas

- **Códigos HTTP esperados:** 400 (Archivo inválido o faltante), 403 (Acceso denegado por rol), 404 (Proveedor no encontrado), 409 (Factura duplicada).
- **Seguridad:** Las rutas están protegidas por el middleware `authenticate` y requieren obligatoriamente el rol `ADMIN`.
- **Auditoría:** Es responsabilidad del operador revisar los avisos de calidad generados en el Paso 1 antes de confirmar. Los archivos originales deben ser conservados fuera del backend (ej. un *bucket* de almacenamiento) si se requiere trazabilidad física/fiscal completa.

8. Descripción de actividades

A continuación, se detallan las actividades clave realizadas durante el desarrollo del backend del sistema Proformax, organizadas cronológicamente y detallando los res-

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

ponsables y medios de verificación correspondientes. El desarrollo se llevó a cabo entre el 01 de abril de 2026 y el 20 de mayo de 2026, siguiendo un enfoque ágil.

Actividad	Responsable	Fecha Inicio	Fecha Fin	Medio verificable
Diseño de la arquitectura en capas del backend (MVC / Arquitectura Hexagonal simplificada) y estructura de carpetas del proyecto (<code>src/</code> , <code>prisma/</code> , <code>logs/</code>).	Chávez Meneses Michael Ricardo (Desarrollador FullStack)	01/04/2026	03/04/2026	Anexo A1: Captura de la estructura de carpetas y diagrama de arquitectura de capas.
Definición del esquema de base de datos relacional con Prisma ORM, aplicación de migraciones (<code>prisma migrate</code>) y creación del script de datos de semilla (<code>prisma:seed</code>).	Chávez Meneses Michael Ricardo (Desarrollador FullStack)	04/04/2026	06/04/2026	Anexo A2: Captura del archivo <code>schema.prisma</code> y del historial de migraciones generadas.
Configuración inicial del servidor Express, implementación de middlewares globales (Helmet, CORS, body-parser) e implementación del módulo de Autenticación con JWT.	Chávez Meneses Michael Ricardo (Desarrollador FullStack)	07/04/2026	12/04/2026	Anexo A3: Captura de la configuración de Express y documentación en Swagger del endpoint de login (<code>swagger_auth.png</code>).
Implementación de los controladores y servicios para los módulos de gestión de Usuarios (RBAC), Clientes y Proveedores.	Chávez Meneses Michael Ricardo (Desarrollador FullStack)	13/04/2026	15/04/2026	Anexo A4: Capturas de Swagger para los módulos Usuarios, Clientes y Proveedores.
Implementación de los controladores y servicios para los módulos de Inventario de Productos, Gestión de Proformas y Configuración de parámetros de la empresa.	Giler Rizzo Neris David (Gestor de Proyecto / Desarrollador)	16/04/2026	06/05/2026	Anexo A5: Capturas de Swagger para los módulos Productos y Proformas.

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

Implementación del módulo de Facturas con integración a Azure AI Document Intelligence, configuración de logs de auditoría y finalización de la documentación Swagger/OpenAPI.	Chávez Meneses Michael Ricardo (Desarrollador FullStack)	07/05/2026	20/05/2026	Anexo A6: Capturas de Swagger del módulo Facturas y del log de auditoría del sistema.
--	---	------------	------------	--

Cuadro 2: Cronograma y descripción de actividades del desarrollo del backend

9. Firmas de Responsabilidad

Acción	Nombre y cargo	Firma	Fecha
Elaborado por:	Chávez Meneses Michael Ricardo (Desarrollador FullStack)		04/05/2026
Revisado por:	Velastegui Eugenio Anthony Paul (Control de Calidad/Documentador)		04/05/2026
Aprobado por:	Giler Risso Neris David (Gestor de Proyecto)		05/05/2026

10. Bibliografía

Referencias

- [1] Organización Internacional de Normalización (ISO), *Sistemas de gestión de la calidad — Requisitos*, ISO 9001:2015, 2015.
- [2] Organización Internacional de Normalización (ISO) / Comisión Electrotécnica Internacional (IEC), *Seguridad de la información, ciberseguridad y protección de la privacidad — Sistemas de gestión de la seguridad de la información — Requisitos*, ISO/IEC 27001:2022, 2022.
- [3] Organización Internacional de Normalización (ISO) / Comisión Electrotécnica Internacional (IEC), *Seguridad de la información, ciberseguridad y protección de la privacidad — Controles de seguridad de la información*, ISO/IEC 27002:2022, 2022.

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

- [4] Asamblea Nacional del Ecuador, *Ley Orgánica de Protección de Datos Personales*, Registro Oficial Suplemento 459, Quito, Ecuador, 26 de mayo de 2021.
- [5] Open Web Application Security Project (OWASP), *Application Security Verification Standard (ASVS) 4.0*, 2021. [En línea]. Disponible en: <https://owasp.org/www-project-application-security-verification-standard/>
- [6] Open Web Application Security Project (OWASP), *OWASP Top 10:2021*, 2021. [En línea]. Disponible en: <https://owasp.org/Top10/>
- [7] Organización Internacional de Normalización (ISO), *Gestión del riesgo — Directrices*, ISO 31000:2018, 2018.
- [8] Organización Internacional de Normalización (ISO), *Seguridad y resiliencia — Sistemas de gestión de la continuidad del negocio — Requisitos*, ISO 22301:2019, 2019.
- [9] National Institute of Standards and Technology (NIST), *Implementation of DevSecOps for a Microservices-based Application with Service Mesh*, NIST Special Publication 800-204C, 2022.
- [10] K. Schwaber y J. Sutherland, *La Guía de Scrum*, 2020. [En línea]. Disponible en: <https://scrumguides.org/>
- [11] Consejo de Europa, *Convenio 108+: Convenio para la protección de las personas con respecto al tratamiento automatizado de datos de carácter personal*, 2018.
- [12] Prisma Data, Inc., *Prisma Documentation*, 2026. [En línea]. Disponible en: <https://www.prisma.io/docs>
- [13] OpenJS Foundation, *Node.js Documentation*, 2026. [En línea]. Disponible en: <https://nodejs.org/docs/>
- [14] Express.js, *Express Application Reference*, 2026. [En línea]. Disponible en: <https://expressjs.com/>

11. Anexos

Anexo A1: Diseño de Arquitectura y Estructura del Proyecto

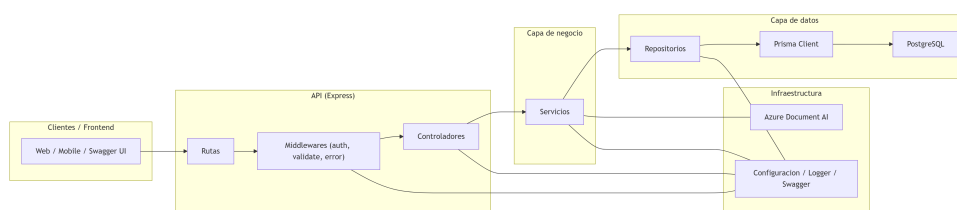


Figura 10: Anexo A1.1 - Diagrama de arquitectura en capas del backend (MVC / Hexagonal)

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

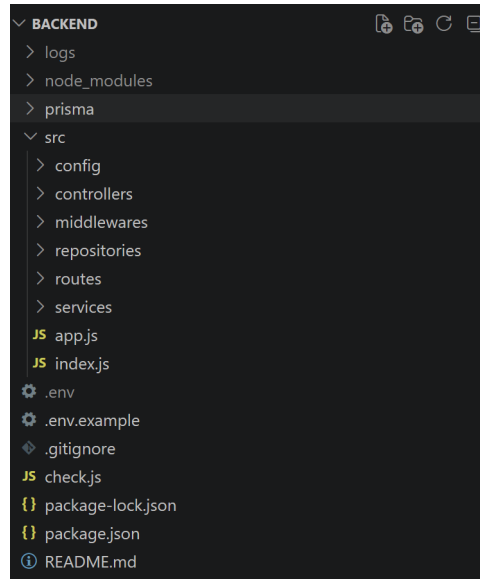


Figura 11: Anexo A1.2 - Captura de la estructura de carpetas en el IDE

Anexo A2: Esquema de Base de Datos y Migraciones

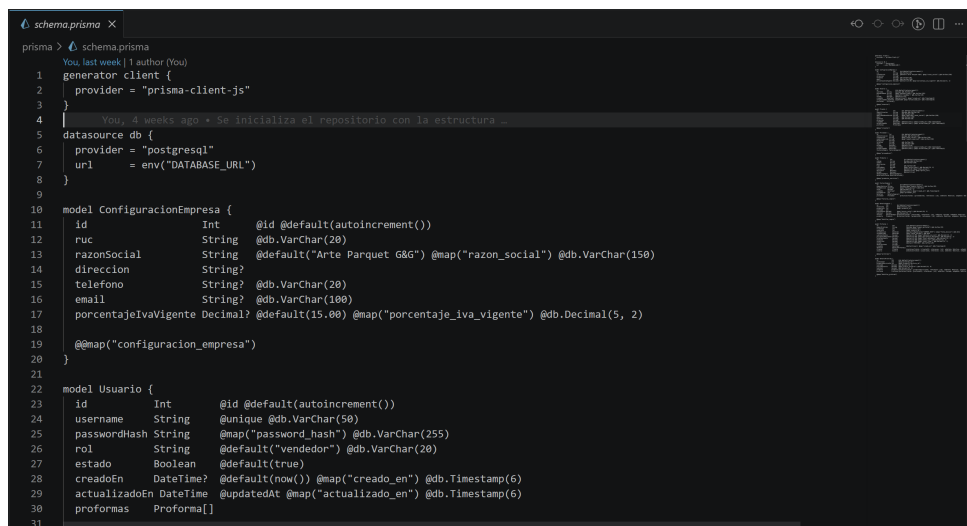


Figura 12: Anexo A2.1 - Definición del esquema relacional en schema.prisma

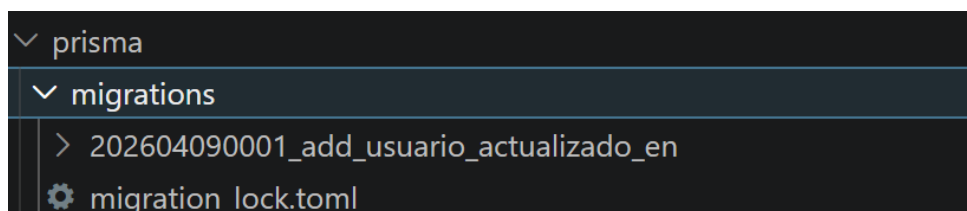


Figura 13: Anexo A2.2 - Historial de migraciones aplicadas en la base de datos

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

Anexo A3: Configuración del Servidor y Autenticación JWT

```

15 app.js
src > $S app.js > ...
21 const app = express();
22
23 // --- Seguridad (ISO/IEC 25010 - LOPDP) ---
24 app.use(helmet({
25   contentSecurityPolicy: {
26     directives: {
27       defaultSrc: ["'self'"],
28       scriptSrc: ["'self'", "unsafe-inline"],
29       styleSrc: ["'self'", "unsafe-inline"],
30       imgSrc: ["'self'", 'data:'],
31     },
32   },
33 }));
34
35 // --- CORS ---
36 const allowedOrigins = (process.env.CORS_ALLOWED_ORIGINS || '').split(',');
37 app.use(cors({
38   origin: (origin, callback) => {
39     if (!origin || allowedOrigins.includes(origin)) return callback(null, true);
40     callback(new Error("CORS bloqueado para origen: ${origin}"));
41   },
42   credentials: true,
43   methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
44   allowedHeaders: ['Content-Type', 'Authorization'],
45   exposedHeaders: ['X-Total-Count'],
46 }));
47
48 // --- Rate Limiting ---
49 const limiter = ratelimit({
50   windowMs: 15 * 60 * 1000, // 15 minutos
51   max: 100,
52   message: { success: false, message: 'Demasiadas peticiones. Intente más tarde.' },

```

Figura 14: Anexo A3.1 - Configuración inicial de Express y middlewares globales

Auth Autenticación y gestión de sesión		
POST	/auth/login	Iniciar sesión
POST	/auth/logout	Cerrar sesión
GET	/auth/me	Perfil del usuario autenticado

Figura 15: Anexo A3.2 - Pruebas del endpoint de Login y validación JWT en Swagger

Anexo A4: Módulos de Usuarios, Clientes y Proveedores

Usuarios Administración de usuarios del sistema (requiere rol ADMIN)		
GET	/usuarios	Listar usuarios
POST	/usuarios	Crear usuario
GET	/usuarios/{id}	Obtener usuario por ID
PUT	/usuarios/{id}	Actualizar datos de usuario
PATCH	/usuarios/{id}/estado	Activar o desactivar usuario

Figura 16: Anexo A4.1 - Documentación y pruebas del módulo Usuarios

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

Clientes Gestión de clientes			^
GET	/clientes	Listar clientes	🔒 ▼
POST	/clientes	Crear cliente	🔒 ▼
GET	/clientes/{id}	Obtener cliente por ID	🔒 ▼
PUT	/clientes/{id}	Actualizar cliente (reemplazo completo)	🔒 ▼
PATCH	/clientes/{id}	Actualizar cliente parcialmente	🔒 ▼
DELETE	/clientes/{id}	Eliminar cliente	🔒 ▼

Figura 17: Anexo A4.2 - Documentación y pruebas del módulo Clientes

Proveedores Gestión de proveedores			^
GET	/proveedores	Listar proveedores	🔒 ▼
POST	/proveedores	Crear proveedor	🔒 ▼
GET	/proveedores/{id}	Obtener proveedor por ID	🔒 ▼
PUT	/proveedores/{id}	Actualizar proveedor completo	🔒 ▼
PATCH	/proveedores/{id}	Actualizar proveedor parcialmente	🔒 ▼
DELETE	/proveedores/{id}	Desactivar proveedor (baja lógica)	🔒 ▼

Figura 18: Anexo A4.3 - Documentación y pruebas del módulo Proveedores

Anexo A5: Módulos de Productos y Proformas

Productos Gestión de productos y servicios (inventario)			^
GET	/productos	Listar productos	🔒 ▼
POST	/productos	Crear producto	🔒 ▼
GET	/productos/{id}	Obtener producto por ID	🔒 ▼
PUT	/productos/{id}	Actualizar producto completo	🔒 ▼
PATCH	/productos/{id}	Actualizar producto parcialmente	🔒 ▼
DELETE	/productos/{id}	Desactivar producto (baja lógica)	🔒 ▼

Figura 19: Anexo A5.1 - Documentación y pruebas del módulo de Inventario de Productos

	EMPRESA DE DESARROLLO E INNOVACIÓN WAVE DEV	CÓDIGO: DE-DEV-SET-002
	PROCESO: FASE 3 DESARROLLO	VERSIÓN: 1.0
	Desarrollo del Backend	ELABORACIÓN: 05/05/2026
	Informe técnico del Desarrollo de Backend Proformax	ÚLTIMA REVISIÓN: 05/05/2026

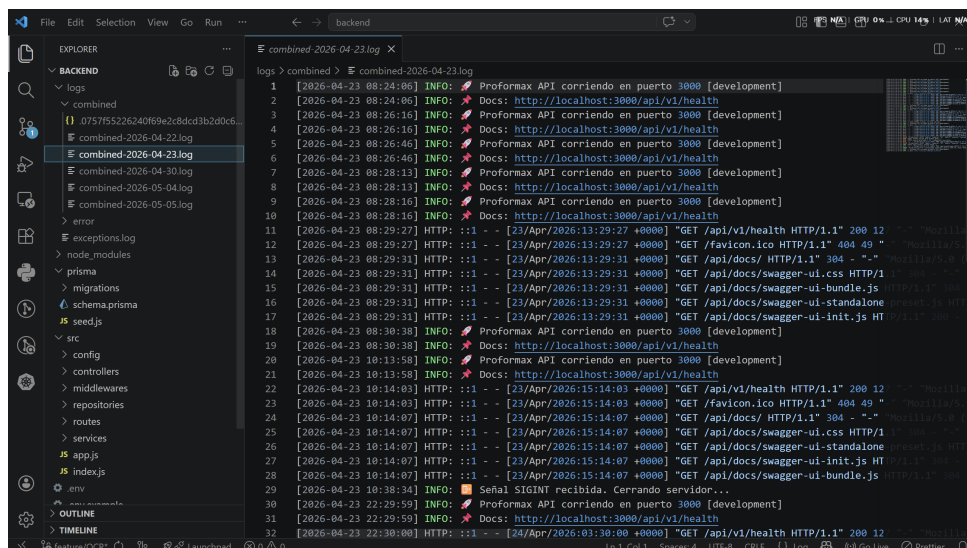
Proformax Creación y gestión del ciclo de vida de proformas comerciales		
GET	/proformas	Listar proformas
POST	/proformas	Crear proforma
GET	/proformas/{id}	Obtener proforma por ID
PUT	/proformas/{id}	Actualizar proforma
GET	/proformas/{id}/pdf	Exportar proforma como PDF
PATCH	/proformas/{id}/estado	Cambiar estado de proforma

Figura 20: Anexo A5.2 - Documentación y pruebas del módulo de Proformas

Anexo A6: Módulo de Facturas e Integración IA / Logs

Facturas Gestión de facturas de compra con Azure AI Document Intelligence. Flujo de dos pasos para garantizar asociación correcta de proveedor.		
GET	/facturas	Listar facturas de compra
GET	/facturas/{id}	Obtener factura por ID con detalles
POST	/facturas/analizar	[Paso 1] Analizar factura con Azure AI (sin guardar)
POST	/facturas/confirmar	[Paso 2] Confirmar y guardar factura

Figura 21: Anexo A6.1 - Documentación y pruebas del módulo Facturas (Integración Azure AI)



```

logs > combined > combined-2026-04-23.log
1 [2026-04-23 08:24:06] INFO: Proformax API corriendo en puerto 3000 [development]
2 [2026-04-23 08:24:06] INFO: Docs: http://localhost:3000/api/v1/health
3 [2026-04-23 08:26:16] INFO: Proformax API corriendo en puerto 3000 [development]
4 [2026-04-23 08:26:16] INFO: Docs: http://localhost:3000/api/v1/health
5 [2026-04-23 08:26:46] INFO: Proformax API corriendo en puerto 3000 [development]
6 [2026-04-23 08:26:46] INFO: Docs: http://localhost:3000/api/v1/health
7 [2026-04-23 08:28:13] INFO: Proformax API corriendo en puerto 3000 [development]
8 [2026-04-23 08:28:13] INFO: Docs: http://localhost:3000/api/v1/health
9 [2026-04-23 08:28:16] INFO: Proformax API corriendo en puerto 3000 [development]
10 [2026-04-23 08:28:16] INFO: Docs: http://localhost:3000/api/v1/health
11 [2026-04-23 08:29:27] HTTP: ::1 - - [23/Apr/2026:13:29:27 +0000] "GET /api/v1/health HTTP/1.1" 200 12 "-" "Mozilla/5.0"
12 [2026-04-23 08:29:27] HTTP: ::1 - - [23/Apr/2026:13:29:27 +0000] "GET /favicon.ico HTTP/1.1" 404 49 "-" "Mozilla/5.0"
13 [2026-04-23 08:29:31] HTTP: ::1 - - [23/Apr/2026:13:29:31 +0000] "GET /api/docs/ HTTP/1.1" 304 - "-" "Mozilla/5.0"
14 [2026-04-23 08:29:31] HTTP: ::1 - - [23/Apr/2026:13:29:31 +0000] "GET /api/docs/swagger-ui.css HTTP/1.1" 200 12 "-" "Mozilla/5.0"
15 [2026-04-23 08:29:31] HTTP: ::1 - - [23/Apr/2026:13:29:31 +0000] "GET /api/docs/swagger-ui-bundle.js HTTP/1.1" 200 12 "-" "Mozilla/5.0"
16 [2026-04-23 08:29:31] HTTP: ::1 - - [23/Apr/2026:13:29:31 +0000] "GET /api/docs/swagger-ui-standalone-preinit.js HTTP/1.1" 200 12 "-" "Mozilla/5.0"
17 [2026-04-23 08:29:31] HTTP: ::1 - - [23/Apr/2026:13:29:31 +0000] "GET /api/docs/swagger-ui-init.js HTTP/1.1" 200 12 "-" "Mozilla/5.0"
18 [2026-04-23 08:30:38] INFO: Proformax API corriendo en puerto 3000 [development]
19 [2026-04-23 08:30:38] INFO: Docs: http://localhost:3000/api/v1/health
20 [2026-04-23 10:13:58] INFO: Proformax API corriendo en puerto 3000 [development]
21 [2026-04-23 10:13:58] INFO: Docs: http://localhost:3000/api/v1/health
22 [2026-04-23 10:14:03] HTTP: ::1 - - [23/Apr/2026:15:14:03 +0000] "GET /api/v1/health HTTP/1.1" 200 12 "-" "Mozilla/5.0"
23 [2026-04-23 10:14:03] HTTP: ::1 - - [23/Apr/2026:15:14:03 +0000] "GET /favicon.ico HTTP/1.1" 404 49 "-" "Mozilla/5.0"
24 [2026-04-23 10:14:07] HTTP: ::1 - - [23/Apr/2026:15:14:07 +0000] "GET /api/docs/ HTTP/1.1" 304 - "-" "Mozilla/5.0"
25 [2026-04-23 10:14:07] HTTP: ::1 - - [23/Apr/2026:15:14:07 +0000] "GET /api/docs/swagger-ui.css HTTP/1.1" 200 12 "-" "Mozilla/5.0"
26 [2026-04-23 10:14:07] HTTP: ::1 - - [23/Apr/2026:15:14:07 +0000] "GET /api/docs/swagger-ui-standalone-preinit.js HTTP/1.1" 200 12 "-" "Mozilla/5.0"
27 [2026-04-23 10:14:07] HTTP: ::1 - - [23/Apr/2026:15:14:07 +0000] "GET /api/docs/swagger-ui-init.js HTTP/1.1" 200 12 "-" "Mozilla/5.0"
28 [2026-04-23 10:14:07] HTTP: ::1 - - [23/Apr/2026:15:14:07 +0000] "GET /api/docs/swagger-ui-bundle.js HTTP/1.1" 200 12 "-" "Mozilla/5.0"
29 [2026-04-23 10:38:34] INFO: Señal SIGINT recibida. Cerrando servidor...
30 [2026-04-23 22:29:59] INFO: Proformax API corriendo en puerto 3000 [development]
31 [2026-04-23 22:29:59] INFO: Docs: http://localhost:3000/api/v1/health
32 [2026-04-23 22:30:00] HTTP: ::1 - - [24/Apr/2026:03:30:00 +0000] "GET /api/v1/health HTTP/1.1" 200 12 "-" "Mozilla/5.0"

```

Figura 22: Anexo A6.2 - Captura del log de auditoría del sistema backend